State Estimation Project Report

Cindy Nguyen

Task 1

Q. With sigma = 0.05 (the standard deviation of the Gaussian), please report the weights of all particles and the index of the most likely particle. Similarly, by setting sigma = 0.1 and sigma = 0.5, repeat the experiment and report the particle weights and the index of the most likely particle. Provide your analysis of how the value of sigma would affect or not affect the particles.

sigma (σ)	Particle weights	Most likely particle
0.05	[3.40644812e-04 2.16563930e-02 1.97339622e-03 1.59178275e-08 2.36661324e-01 7.39063591e-01 1.41646902e-04 1.62988550e-04 6.71746152e-34 1.81168510e-51]	5
0.1	[5.24895495e-02 1.48215605e-01 8.14331011e-02 4.33978635e-03 2.69481666e-01 3.58234866e-01 4.21501545e-02 4.36552696e-02 1.96697435e-09 7.97109672e-14]	5
0.5	[0.11188281 0.11662622 0.11386559 0.10126496 0.11944875 0.12081675 0.11090533 0.11106108 0.0564563 0.03767222]	5

How the value of sigma affects the particles:

In the probability density function, sigma represents the standard deviation of the Gaussian distribution, which determines our confidence in the sensor measurements.

A smaller sigma (closer to 0.05) creates a tall, narrow Gaussian with a higher peak. This makes the filter very discriminative, and particles with predicted observations that don't closely match the actual observation are severely penalized (assigned weights near zero). While this leads to sharper, faster convergence, it risks particle collapse if no particles happen to be very close to the ground truth.

A larger sigma (closer to 0.5) creates a wide, flat Gaussian with a lower peak. This makes the filter more forgiving to errors and noise. Particles farther from the ground truth will still receive non-zero weights. While this increases robustness against poor initial particle distributions, it reduces the filter's precision and significantly slows down convergence because bad particles aren't eliminated fast enough.

Task 2

Q. With 1000 and 2500 particles, and with sigma = 0.01, 0.05, 0.2 and delta=0.001, 0.01, 0.1 respectively, run your implemented Particle Filtering algorithm. For each combination of sigma, delta and the number of particles, you can run multiple trials. In the report:

1. discuss how the number of particles affect the performance of the Particle Filtering algorithm (e.g., the convergence speed, the robustness, etc)
2. discuss how the value of sigma affect the performance of the Particle Filtering algorithm (e.g., the convergence speed, the robustness, etc)
3. discuss how the value of delta affect the performance of the Particle Filtering algorithm (e.g., the convergence speed, the robustness, etc)
4. report your estimate of the pose, i.e., (x, y, θ) of the robot base

Combination (num_particles, sigma, delta)	Trial 1	Trial 2	Trial 3	Average
1000, 0.01, 0.001	[0.07388563 0.11259171 1.55918724]	[0.01501129 -0.86705388 0.39070523]	[0.29530955 -0.50024272 -1.41694925]	[0.12806882 -0.41823496 0.17764774]
1000, 0.01, 0.01	[0.10860222 0.07826182 1.52683206]	[0.05267163 0.07838924 1.4528718]	[0.14719798 0.88025087 -1.26290022]	[0.10282394 0.34563398 0.57226788]
1000, 0.01, 0.1	[1.087047 -0.09248072 -2.91950784]	[-0.05774403 -0.04989333 1.50324671]	[-0.17949093 -0.1631273 1.27625911]	[0.28327068 -0.10183378 -0.04666734]
1000, 0.05, 0.001	[0.11444417 0.10759828 1.64748119]	[-0.88320318 -0.04155689 1.64841216]	[0.26573813 -0.50032454 -1.37581909]	[-0.16767363 -0.14476105 0.64002475]
1000, 0.05, 0.01	[0.29594108 -0.71244401 0.52747528]	[0.02541159 -0.67398144 -0.13918082]	[-0.29307567 -0.30051888 0.90672478]	[0.00942567 -0.56231478 0.43167308]
1000, 0.05, 0.1	[-0.26946035 -0.22046592 0.97239212]	[-0.23986475 -0.13982748 1.19291199]	[-0.25651481 -0.16267528 1.22433751]	[-0.25527997 -0.17432289 1.12988054]
1000, 0.2, 0.001	[0.31441289 -0.55902092 -1.29988242]	[-0.03887084 -0.69725944 0.87244059]	[0.03029174 -0.26647912 0.46923754]	[0.10194460 -0.50758649 0.01393190]
1000, 0.2, 0.01	[0.06263819 -0.61402668 -0.26605479]	[0.36877003 -0.41999323 -1.39936788]	[-0.34107498 -0.2449309 0.77503105]	[0.03011108 -0.42631694 -0.29679721]

1000, 0.2, 0.1	[-0.0914784 -0.11873515 1.52284217]	[-0.13440732 -0.05655584 0.64541002]	[-0.10814576 -0.12051204 1.30353612]	[-0.11134383 -0.09860101 1.15726277]
2500, 0.01, 0.001	[0.56972045 -0.8974187 2.81976741]	[0.90175386 0.06461344 -2.18865947]	[0.49331435 -1.01983326 2.403099]	[0.65492955 -0.61754617 1.01140231]
2500, 0.01, 0.01	[0.11983324 0.81761892 -1.5023251]	[-0.03297659 0.19806379 -1.26131698]	[-0.2993864 -0.29999995 0.90131085]	[-0.07084325 0.23856092 -0.62077708]
2500, 0.01, 0.1	[-0.20369879 -0.16646319 1.41209403]	[-0.16950402 -0.12926277 1.3287702]	[-0.24234315 -0.17379199 1.23884079]	[-0.20518199 -0.15650598 1.32656834]
2500, 0.05, 0.001	[-0.25724483 -0.28278741 0.92313948]	[-0.32013746 -0.30246425 0.93656416]	[-0.23844694 -0.32926058 1.02498526]	[-0.27194308 -0.30483741 0.96156297]
2500, 0.05, 0.01	[-0.29831511 -0.30079656 0.90365654]	[-0.30918338 -0.28832555 0.87261829]	[-0.29829197 -0.29759376 0.90026708]	[-0.30193015 -0.29557196 0.89218064]
2500, 0.05, 0.1	[-0.24689382 -0.1673001 1.15854922]	[-0.23793785 -0.15572446 1.25846089]	[-0.2497649 -0.15751824 1.2069969]	[-0.24486552 -0.16018093 1.20800234]
2500, 0.2, 0.001	[0.02269514 -0.35271023 -0.31229716]	[0.35701855 -0.66860803 -0.75867272]	[0.23214459 -0.52734143 1.02477619]	[0.20395276 -0.51621990 -0.01539790]
2500, 0.2, 0.01	[0.18511467 -0.42181304 -0.56701641]	[0.27687685 -0.65939485 0.06752995]	[0.03997987 -0.51579084 -0.27738674]	[0.16732380 -0.53233291 -0.25895773]
2500, 0.2, 0.1	[-0.10357214 -0.06241429 0.95317258]	[-0.12580879 -0.11407251 1.15731808]	[-0.08883842 -0.09167721 1.08355642]	[-0.11469046 -0.08824340 1.05524533]

Robot Base Pose Estimate: Ground truth is at [-0.3, -0.3, 0.90]. My best estimate is at [-0.30193015, -0.29557196, 0.89218064] for num_particles=2500, sigma=0.05, delta=0.01.

How the Hyperparameters affect the performance of the Particle Filtering algorithm:

- **Number of Particles (1000, 2500):** Increasing the number of particles significantly improves the robustness and accuracy of the filter, because a denser sampling provides better coverage of the state space. This reduces the chance of the filter getting stuck in a local optimum. However, this has a direct cost of computational speed. 1000 particles generally offer a balanced trade-off, converging not as accurately but with less lag.

- **Sigma (0.01, 0.05, 0.2):** Similar to Task 1, sigma controls the Gaussian distribution standard deviation. A low sigma = 0.01 caused fast convergence but suffered from inflexibility. If the ground truth wasn't captured well by the initial random scatter, the weights all collapsed to zero. A high sigma = 0.2 was more robust but the particle cloud was thick and dispersed, resulting in a less precise final estimate. Therefore, the optimal sigma was 0.05.
- **Delta (0.001, 0.01, 0.1):** delta determines the scale of the Gaussian noise added during the perturbation step to prevent particle collapse. A low delta = 0.001 caused resampled particles to basically stack on top of each other, leading to early convergence and a lack of exploration. Meanwhile, a high delta = 0.1 caused the particles to scatter wildly at every iteration, inhibiting the cloud from settling into a tight, confident cluster. The value delta = 0.01 provided enough jitter to explore the local space while maintaining steady convergence.

Task 3

Q. In the report, please describe your online Particle Filtering algorithm (including how you control the robot and collect observations online) with both text and pseudo codes. Please visualize the particles in your report to show the convergence of the particles. You can choose to use any hyperparameters (the number of particles, the number of iterations, the values of sigma and delta) that you believe work the best. Please report the values of these hyperparameters you used in your experiments.

Algorithm Description:

1. **Control the robot to collect an observation online:** The robot randomly explores its environment to seek out constraints. It saves its current safe state and randomly samples a 3D Cartesian translational velocity. It moves iteratively in this direction. If the spherical probe successfully touches an obstacle, the robot records its joint states as a valid observation, reverts to its safe starting pose, and returns the observation. If the robot experiences an unsafe body collision, it aborts, reverts to the safe pose, and attempts a new random direction.
2. **Online Particle Filtering algorithm:** Once an online observation is retrieved, the algorithm follows these steps:
    - Weight Update: Calculates weights for all particles based on the observation.
    - Resampling: Draws a set of particles with replacement according to their weights.
    - Perturbation: Adds small Gaussian noise to avoid particle collapse.
    - This loop repeats to iteratively refine the pose estimate.

Pseudocode:

```
def get_one_obv(panda_sim):
    obv ← None
    state ← panda_sim.save_state()
    while True do
        panda_sim.restore_state(state)
        direction ← np.random.uniform(-1, 1, size=3)
        direction ← direction / norm(direction)
        v ← np.zeros(6)
        v[0:3] ← direction * 0.5
        for _ in range(500) do
            if panda_sim.is_touch() then
                jpos, _, _ ← panda_sim.get_joint_states()
                obv ← array(jpos[0:7])
                panda_sim.restore_state(state)
                return obv
            if panda_sim.is_collision() then
                break
```

```
            panda_sim.execute(v)
    return obv

def particle_filter_online(panda_sim, num_particles, sigma=0.05, delta=0.01,
plot=True):
    particles ← np.random.uniform(low=[-1, -1, -np.pi], high=[1, 1, np.pi],
size=(num_particles, 3))
    weights ← np.ones(shape=(num_particles,)) / num_particles
    num_iters ← 100
    for _ in range(num_iters) do
        obv ← get_one_obv(panda_sim)
        weights ← cal_weights(particles, obv, sigma=sigma)
        idxs ← np.random.choice(num_particles, size=num_particles,
replace=True, p=weights)
        particles ← particles[idxs]
        noise ← np.random.normal(loc=0.0, scale=delta, size=particles.shape)
        particles ← particles + noise
    est ← particles.mean(0)
    return est
```

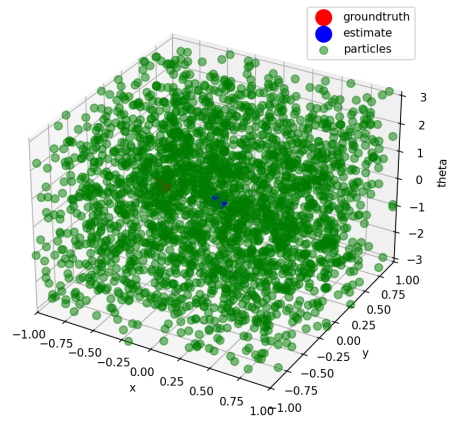
Hyperparameters:

- Number of Particles: 2500
- Iterations: 100
- Sigma: 0.05
- Delta: 0.01

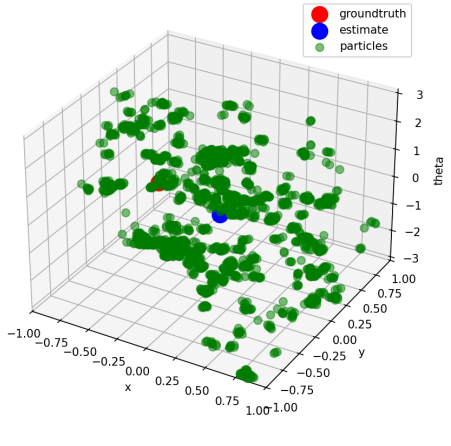
Pose estimate: [-0.30631381 -0.29587831 0.88311913]

Particle Convergence Visualizations:

Initial:



Mid-way:



Final:

